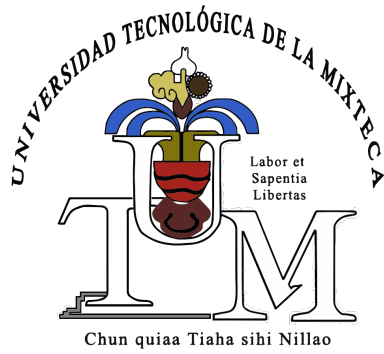




OAXACA

Universidad Tecnológica De La Mixteca

Prof. David Martínez Torres
Compiladores
502-A
Equipo 1
Algoritmo de Primeros y Siguientes

Integrantes:

Lujan De la Rosa Cristian Emir
Cortez Vega Jaime Fabian
Carrasco Pacheco Levi Josue
Nicolas Vasquez Joel Osmar
Vasquez Moreno Zair De Jesus
Arias Mendoza Angel Adrian

Análisis

Los conjuntos **PRIMERO** y **SIGUIENTE** son estructuras fundamentales en el diseño de analizadores sintácticos predictivos (LL(1)). Su cálculo correcto a partir de una gramática libre de contexto determina qué terminal puede iniciar una derivación (**PRIMERO**) y qué terminal puede aparecer inmediatamente a la derecha de un no terminal en alguna forma sentencial (**SIGUIENTE**).

Para el cómputo de ambos conjuntos se adoptó un enfoque iterativo de punto fijo: se aplican las reglas de inferencia de manera repetida hasta que ningún conjunto varía entre dos iteraciones consecutivas. Este método garantiza la convergencia para cualquier gramática libre de contexto finita.

Reglas de PRIMERO

1. Si X es un terminal, entonces $\text{PRIMERO}(X) = \{X\}$.
2. Si existe la producción $X \rightarrow \varepsilon$, se añade ε a $\text{PRIMERO}(X)$.
3. Para cada producción $X \rightarrow Y_1 Y_2 \cdots Y_n$:
 - Se añade $\text{PRIMERO}(Y_1) - \{\varepsilon\}$ a $\text{PRIMERO}(X)$.
 - Si $\varepsilon \in \text{PRIMERO}(Y_i)$ para todo $i < k$, se añade también $\text{PRIMERO}(Y_k) - \{\varepsilon\}$.
 - Si $\varepsilon \in \text{PRIMERO}(Y_i)$ para todo i , se añade ε a $\text{PRIMERO}(X)$.

Reglas de SIGUIENTE

1. $\$ \in \text{SIGUIENTE}(S)$ donde S es el símbolo inicial de la gramática.
2. Para cada producción $A \rightarrow \alpha B \beta$:
 - Se añade $\text{PRIMERO}(\beta) - \{\varepsilon\}$ a $\text{SIGUIENTE}(B)$.
 - Si $\varepsilon \in \text{PRIMERO}(\beta)$ (o $\beta = \varepsilon$), se añade $\text{SIGUIENTE}(A)$ a $\text{SIGUIENTE}(B)$.

El proyecto implementa ambos algoritmos en Python mediante una arquitectura modular dividida en tres capas: modelos, persistencia y presentación. La capa de modelos define la clase **Gramatica**; la capa de persistencia agrupa los módulos de lectura de archivo y de cómputo de los conjuntos; y la capa de presentación contiene la interfaz gráfica construida con Tkinter.

Asignación de actividades

Sub equipos:

1. Cortez Vega Jaime Fabian y Lujan De la Rosa Cristian Emir
2. Arias Mendoza Angel Adrian y Vasquez Moreno Zair De Jesus
3. Carrasco Pacheco Levi Josue y Nicolas Vasquez Joel Osmar

Equipo	Actividad	Entregas clave	Fecha de Entrega
Todos	Diseño de la estructura general del proyecto, definición de la clase <code>Gramatica</code> y comprensión del algoritmo de punto fijo.		Semana 1.
1	Implementación del algoritmo de PRIMERO y del algoritmo de SIGUIENTE mediante iteración de punto fijo.	Módulos <code>algoritmo_primeros.py</code> y <code>algoritmo_siguietes.py</code> funcionales con pruebas sobre gramáticas de ejemplo.	Semana 2.
2	Implementación de la clase <code>Gramatica</code> y del lector de archivos <code>.txt</code> en sus dos formatos (simple y con encabezados).	Módulos <code>gramatica.py</code> y <code>lector_archivo.py</code> con manejo de errores y soporte para ϵ .	Semana 2.
3	Diseño e implementación de la interfaz gráfica: ventana principal, editor de gramática con numeración de líneas y ventana de resultados con pestañas.	Módulos de presentación integrados en <code>interfaz.py</code> ; interfaz funcional y probada en Windows y macOS.	Semana 3.

- Es importante que cada integrante del equipo registre su avance diario en el informe de actividades, describiendo las tareas realizadas o los errores encontrados.

- La primera reunión de control de avances tendrá lugar al finalizar la semana 1, donde los sub equipos presentarán su diseño y resolverán dudas.
- La segunda reunión se realizará al cierre de la semana 2 para integrar los módulos de algoritmos y modelo de datos con la interfaz en desarrollo.
- Los días restantes se dedicarán a pruebas de integración, corrección de errores y presentación final con el profesor.

Clases y métodos implementados por cada subequipo

Cortez Vega y Lujan De la Rosa:

1. **Módulo Algoritmo PRIMERO** (`algoritmo_primeros.py`): Implementa la función `calcular_primeros`, que calcula los conjuntos PRIMERO para todos los símbolos de la gramática mediante un algoritmo iterativo de punto fijo. La función recibe el símbolo de consulta, el diccionario de producciones, un diccionario mutable de conjuntos `primeros` y la lista de terminales. Inicializa cada terminal con su propio conjunto y cada no terminal con el conjunto vacío; luego itera hasta que no haya cambios.
2. **Módulo Algoritmo SIGUIENTE** (`algoritmo_siguietes.py`): Implementa dos funciones:
 - a. **`_primero_secuencia`**: función auxiliar que calcula PRIMERO de una secuencia de símbolos e indica si toda la secuencia puede derivar en ε .
 - b. **`calcular_siguietes`**: función principal que aplica el algoritmo de SIGUIENTE mediante iteración de punto fijo. Inicializa el símbolo inicial con $\{\$ \}$ y, para cada producción $A \rightarrow \alpha B \beta$, propaga $\text{PRIMERO}(\beta)$ y, cuando corresponde, $\text{SIGUIENTE}(A)$ hacia $\text{SIGUIENTE}(B)$.

Arias Mendoza y Vasquez Moreno:

1. **Módulo Gramática** (`gramatica.py`): Define la clase `Gramatica`, que representa una gramática libre de contexto. Sus atributos son: `terminales`, `no_terminales`, `simbolo_inicial` y `producciones` (diccionario de cabeza a lista de cuerpos). El método `agregar_produccion` inserta una nueva regla de producción al diccionario.
2. **Módulo Lector de Archivo** (`lector_archivo.py`): Implementa la función `cargar_gramatica_desde_archivo`, que lee un archivo `.txt` codificado en UTF-8 y construye un objeto `Gramatica`. Soporta dos formatos:
 - a. **Formato simple**: solo producciones; terminales y no terminales se infieren automáticamente cualquier símbolo que encabece una producción es no terminal, el resto son terminales.
 - b. **Formato con encabezados**: las dos primeras líneas declaran explícitamente los terminales y no terminales; las producciones inician en la tercera línea.

Acepta producciones vacías escritas como ε , `e` o `epsilon` (sin distinción de mayúsculas) y lanza `ValueError` ante cualquier producción malformada.

Carrasco Pacheco y Nicolas Vasquez:

1. **Módulo de Estilos** (`estilos.py`): Define la paleta de colores (`COLORES`), la función `configurar_estilos` que aplica los temas `ttk` globalmente, y la función `crear_superficie` que genera marcos con un diseño visual uniforme para todas las secciones de la interfaz.
2. **Módulo Tabla Base** (`tabla_base.py`): Widget `Treeview` reutilizable que incluye barras de desplazamiento vertical y horizontal. Permite configurar las columnas, sus encabezados, anchos y alineación, y es utilizado como base por los demás módulos de presentación.
3. **Módulo Tabla Conjuntos** (`tabla_conjuntos.py`): Especializa la tabla base para mostrar los conjuntos `PRIMERO` o `SIGUIENTE`, presentando un no terminal por fila y su conjunto correspondiente como cadena de símbolos separados por comas.
4. **Módulo Ventana de Resultados** (`ventana_primeros_siguietes.py`): Implementa la clase `VentanaPrimerosSiguietes`, una ventana secundaria con dos pestañas: una para los conjuntos `PRIMERO` y otra para los conjuntos `SIGUIENTE`. Cada pestaña muestra una `TablaConjuntos` con los resultados del análisis.
5. **Módulo Interfaz Principal** (`interfaz.py`): Implementa la clase `InterfazCompilador`, punto de entrada de la aplicación. Proporciona:
 - Carga de archivos `.txt` mediante diálogo del sistema operativo.
 - Editor de solo lectura con numeración de líneas para inspeccionar la gramática.
 - Botón **Analizar** que invoca los módulos de `PRIMERO` y `SIGUIENTE` y abre la ventana de resultados.
 - Botón **Limpiar** para reiniciar el estado de la aplicación.
 - Barra de estado con mensajes informativos sobre el progreso del análisis.

Avances

Conclusiones

Llevar a cabo este proyecto nos permitió comprender de manera práctica el papel de los conjuntos PRIMERO y SIGUIENTE en la construcción de analizadores sintácticos predictivos. A diferencia del análisis léxico que reconoce tokens mediante autómatas finitos el análisis sintáctico predictivo requiere razonar sobre las derivaciones posibles de la gramática, y los conjuntos PRIMERO y SIGUIENTE son la herramienta que hace posible tomar esas decisiones sin necesidad de retroceso.

La implementación del algoritmo iterativo de punto fijo nos exigió comprender a fondo las dependencias entre los conjuntos de distintos no terminales: un cambio en PRIMERO de un símbolo puede propagarse hacia los conjuntos SIGUIENTE de otros. Manejar correctamente estas dependencias mediante la repetición controlada de las reglas de inferencia fue el reto central del proyecto.

Desde el punto de vista del diseño de software, la separación en capas modelos, persistencia y presentación facilitó la división del trabajo entre sub equipos y permitió desarrollar y probar cada módulo de forma independiente antes de la integración. La interfaz gráfica, construida con Tkinter, añadió valor al proyecto al hacer accesible el cálculo de los conjuntos a cualquier usuario sin necesidad de conocer el código.

En conjunto, el proyecto consolidó nuestra comprensión de la segunda fase del diseño de compiladores y reforzó buenas prácticas de colaboración, modularidad y documentación que continuaremos aplicando en etapas posteriores del curso.

Referencias

Martínez Torres, D. (s. f.). *Tema 3. Análisis sintáctico* [Diapositivas de PowerPoint]. Instituto de Computación, Universidad Tecnológica de la Mixteca.

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.1 ed.). Pearson Educación.

Mogensen, T. Æ. (2017). *Introduction to Compiler Design* (2nd ed.). Springer.